Resolution Strategies for Client-Level Disagreement Scenarios in Federated Learning

Daan E. Rosendal
MSc Software Engineering
University of Amsterdam

Academic supervisor: Dr. A.M. (Ana) Oprescu
Second reader: Dr. L.T. (Thomas) van Binsbergen
Research group: Complex Cyber Infrastructure (CCI)

11 July 2025

Table of contents

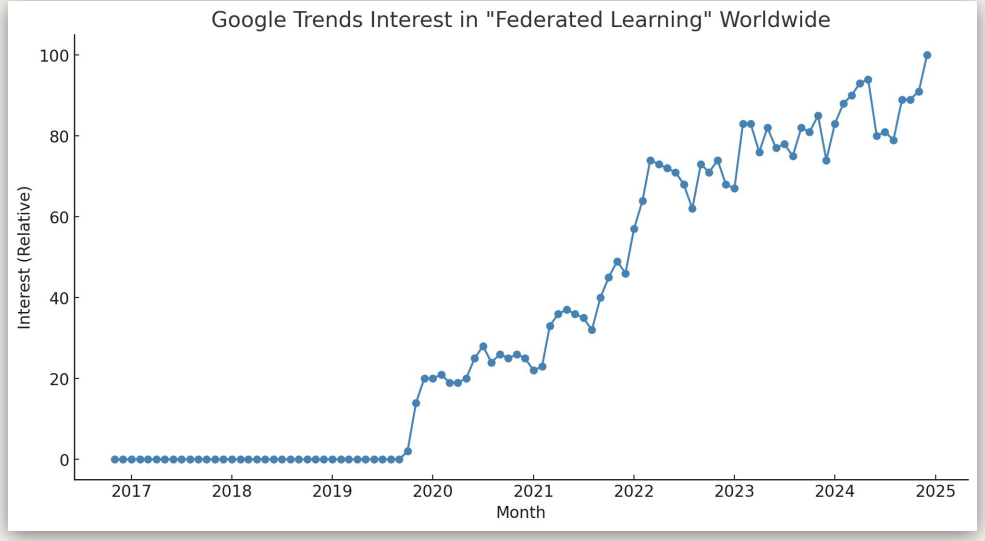
1. Introduction
2. Disagreements scenarios
3. Resolution strategies
4. System design & implementation
5. Experimental setup
6. Results
7. Discussion
8. Conclusion

1

Introduction

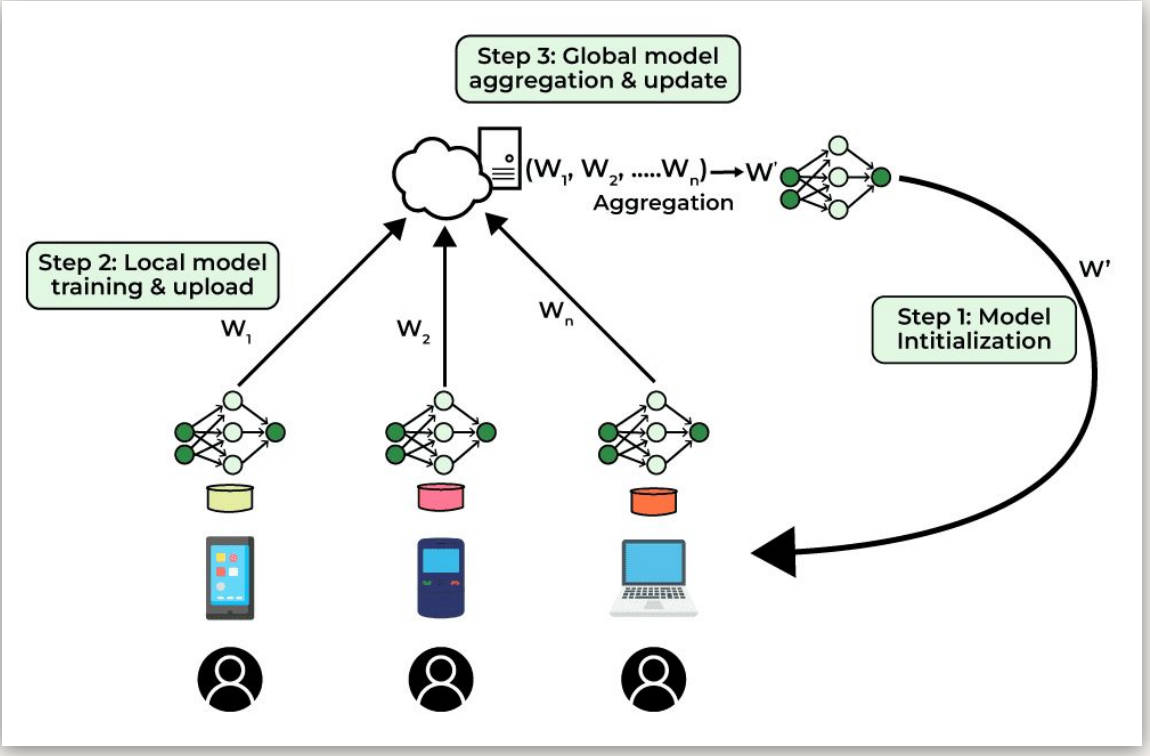
Problem statement

- Research gap
- Data-level disagreements
- Industry interest
- Relevance: FL 🚀



Federated Learning (FL)

- Distributed ML with data privacy
- Federated Averaging [1]
- Main benefit: better model
- Example use cases:
  - Jet engine failure prediction
  - Credit card fraud detection



Source: https://www.geeksforgeeks.org/collaborative-learning-federated-learning

Client-level disagreements

- Client-controlled restrictions on model flow
  - Client might say: “I do not want client X to influence my model!”
- Differs from data-level disagreements:
  - Intentional vs implicit
  - Policy vs statistics
- Why to accommodate such disagreements:
  - Regulatory: comply with data sovereignty
  - Security: exclude suspected adversarial or low-quality clients
  - Strategic: protect competitive advantage

Research questions (shortened)

RQ1: What disagreement scenarios can occur?

RQ2: How can they be resolved?

- **RQ2.1:** How can sustainability be improved for similar disagreements?
- **RQ2.2:** How can granular access control help clients?

RQ3: How can temporal disagreements be accommodated?

- **RQ3.1:** How can clients rejoin the system?

RQ4: How can scalability challenges be accounted for?

Research method

01 Literature review

Identifying the research gap

02 Exploratory study

Disagreement taxonomy and resolution strategies

03 Design & implementation

Building a proof-of-concept

04 Empirical evaluat-ion & validation

Validating correctness and scalability

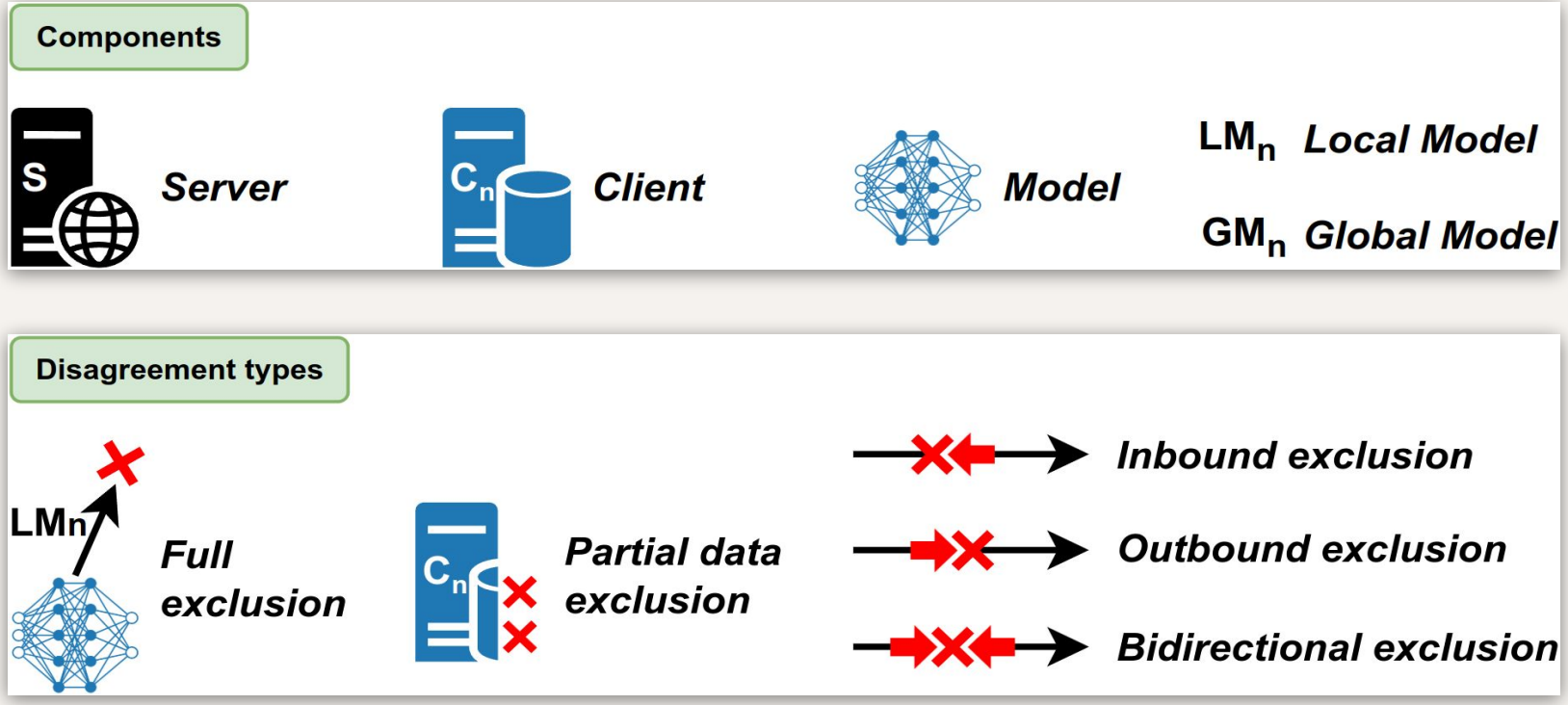
2

Disagreement scenarios

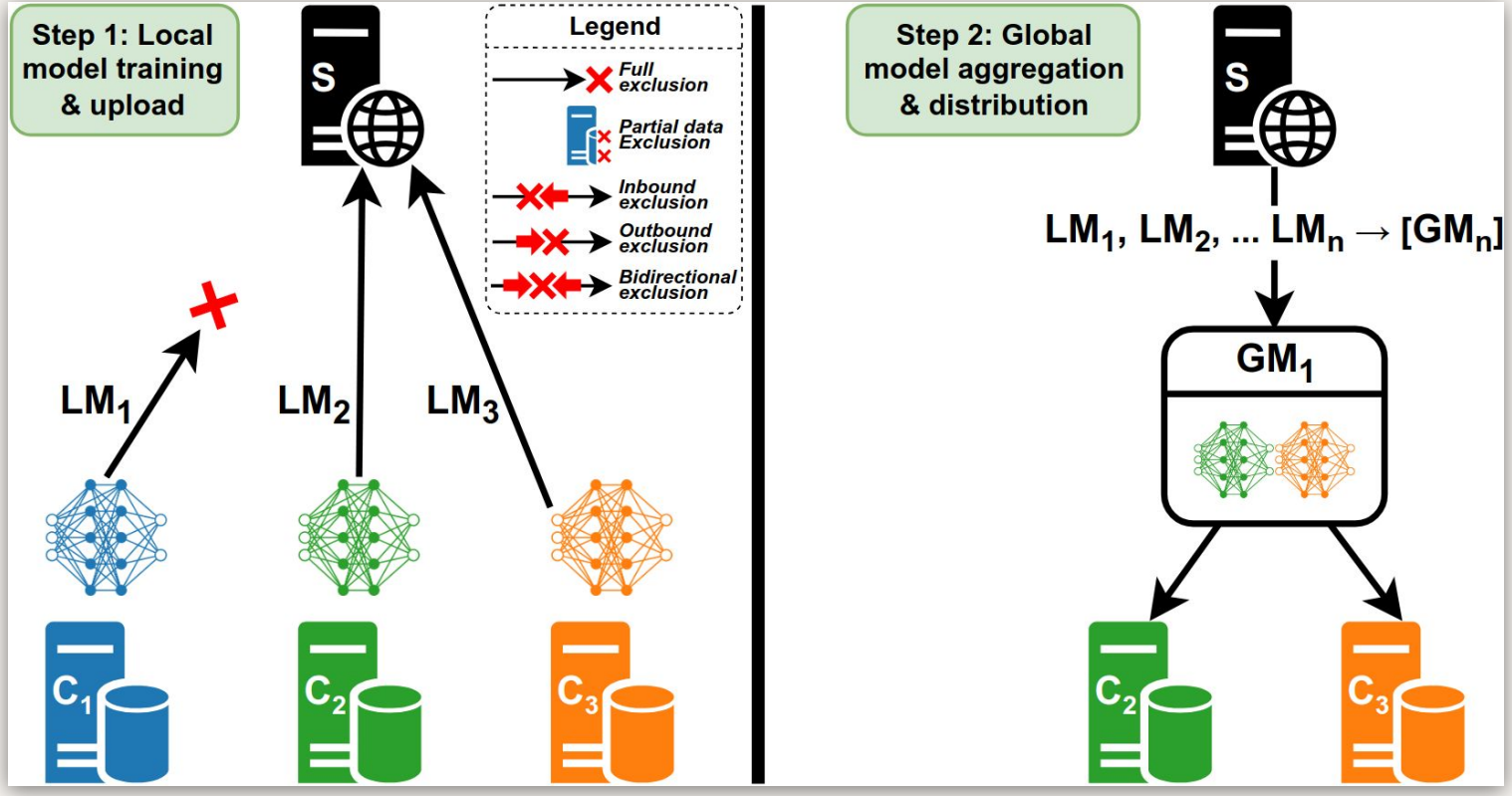
RQ1: what disagreement scenarios can occur?

Exclusion	**Description**	**Category**
Full	Complete withdrawal	Communication, data
Partial data	Withholding specific local data for training	Data
Inbound	Restricting who *affects* my model	Communication
Outbound	Restricting who *receives* my model influence	Communication
Bidirectional	Mutual block on model exchange	Communication
Temporary	Expires after a predefined period	Temporal
Deep	Retroactively removes contributions	Temporal

Legend

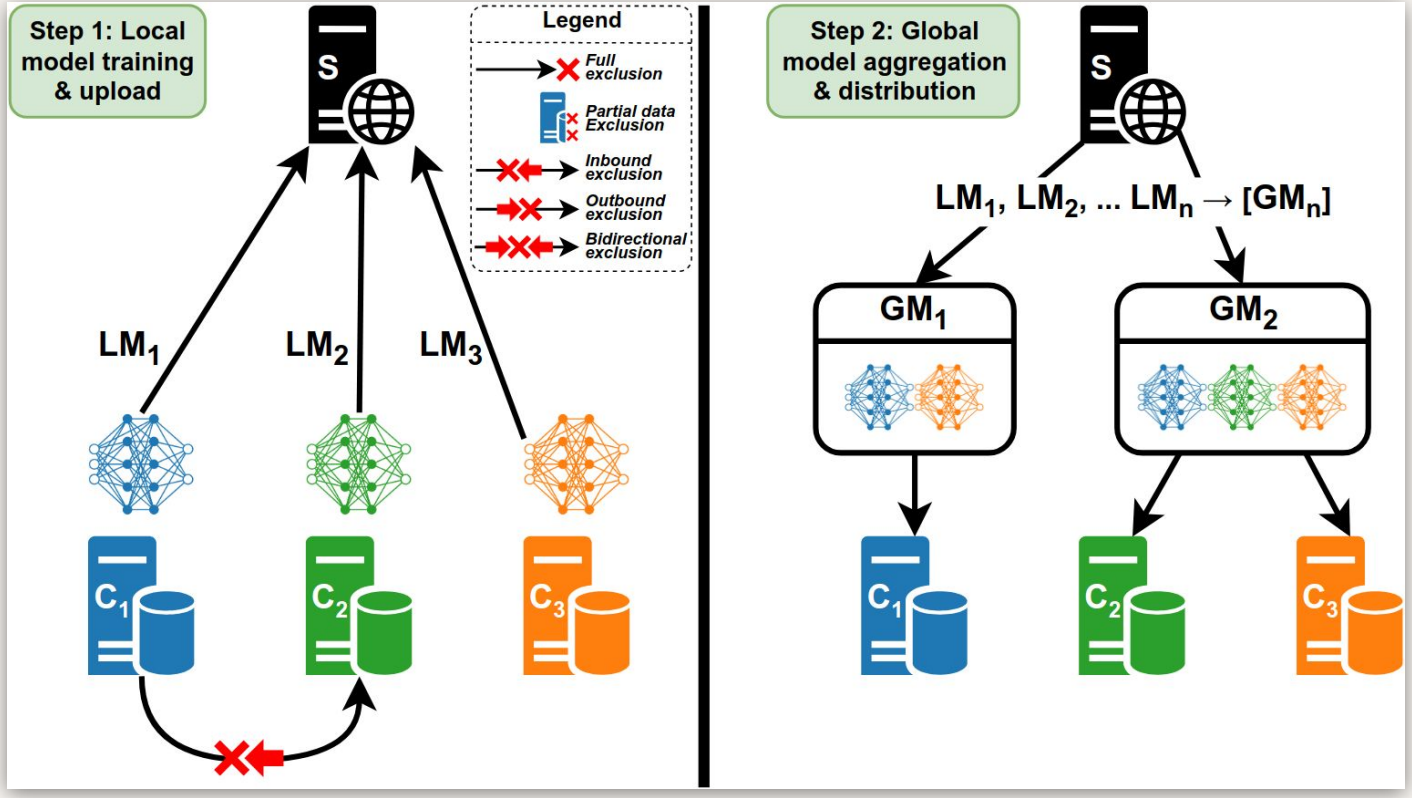

Components
S
Server
C_n
Client
Model
LM_n Local Model
GM_n Global Model
Disagreement types
LMn
Full exclusion
C_n
Partial data exclusion
Inbound exclusion
Outbound exclusion
Bidirectional exclusion

Full exclusion Complete withdrawal



Inbound exclusion

Restricting who *affects* my model

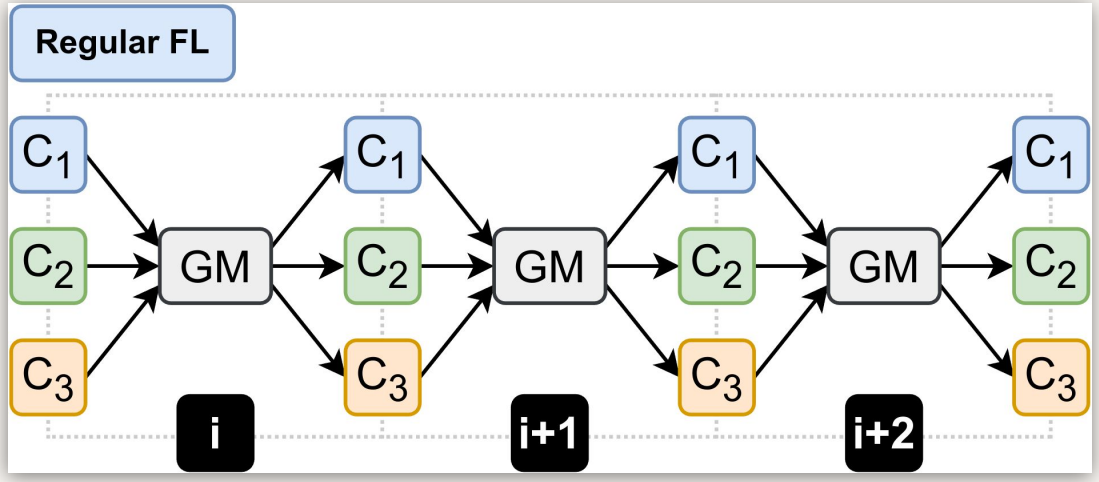


3

Resolution strategies

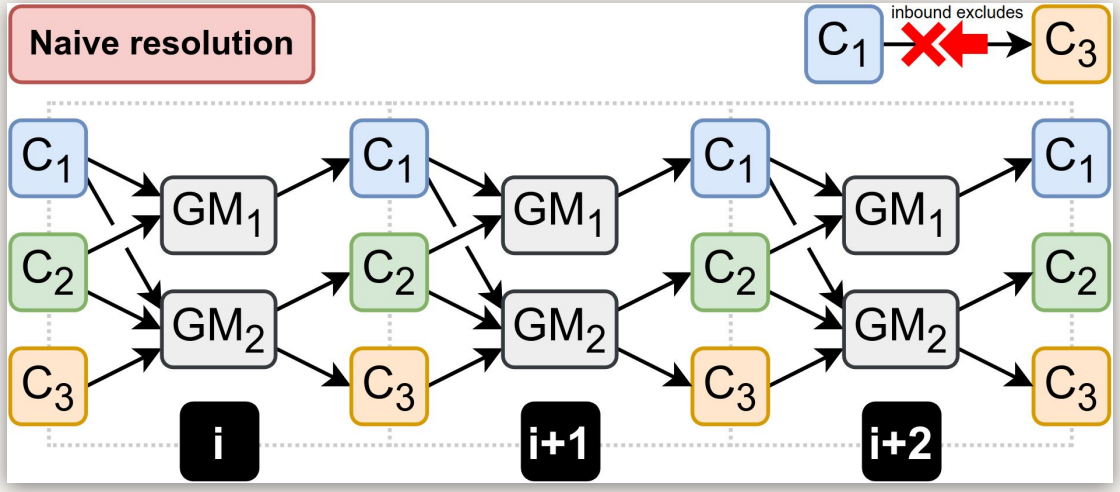
Regular Federated Learning

- Goal: visualising disagreements across rounds



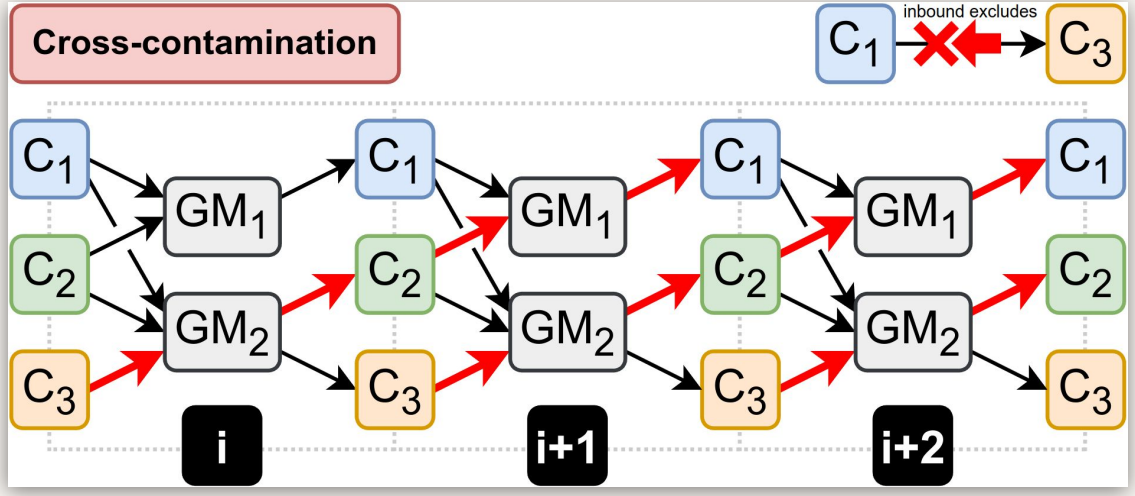
Naive resolution

- C_1 inbound excludes C_3, resulting in 2 GMs



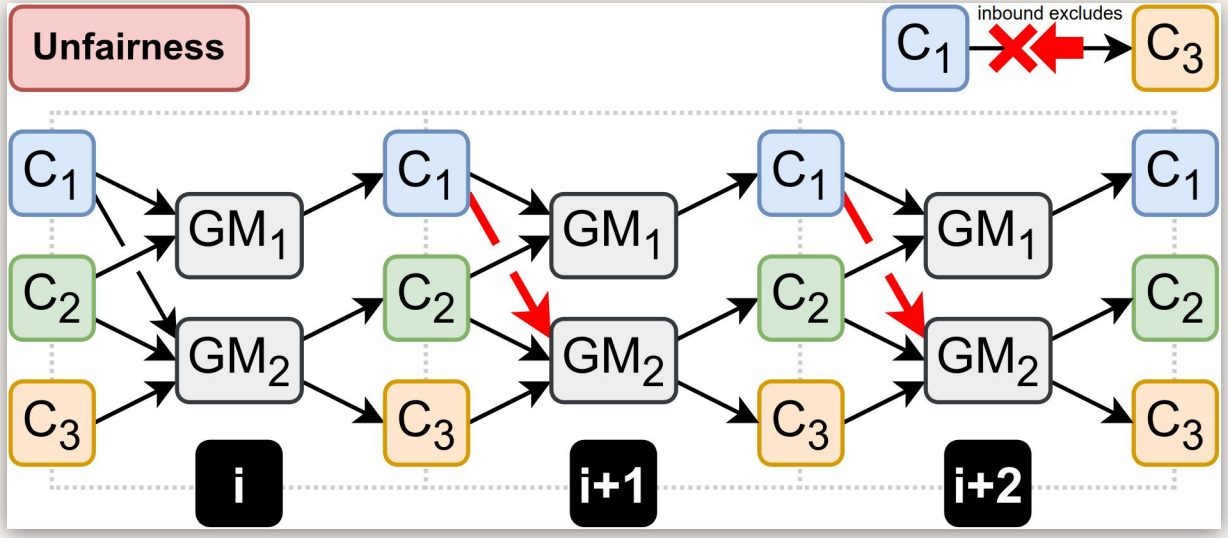
Naive resolution: cross-contamination

- C_2 passes C_3's influence to C_1 across rounds



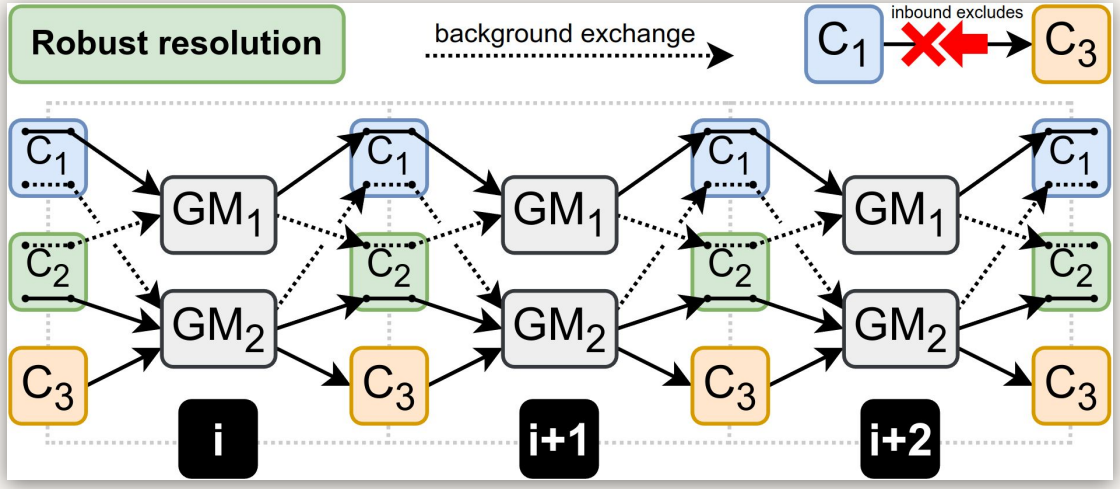
Naive resolution: unfairness

- C_2 and C_3 unfairly receive C_1's irregular influence



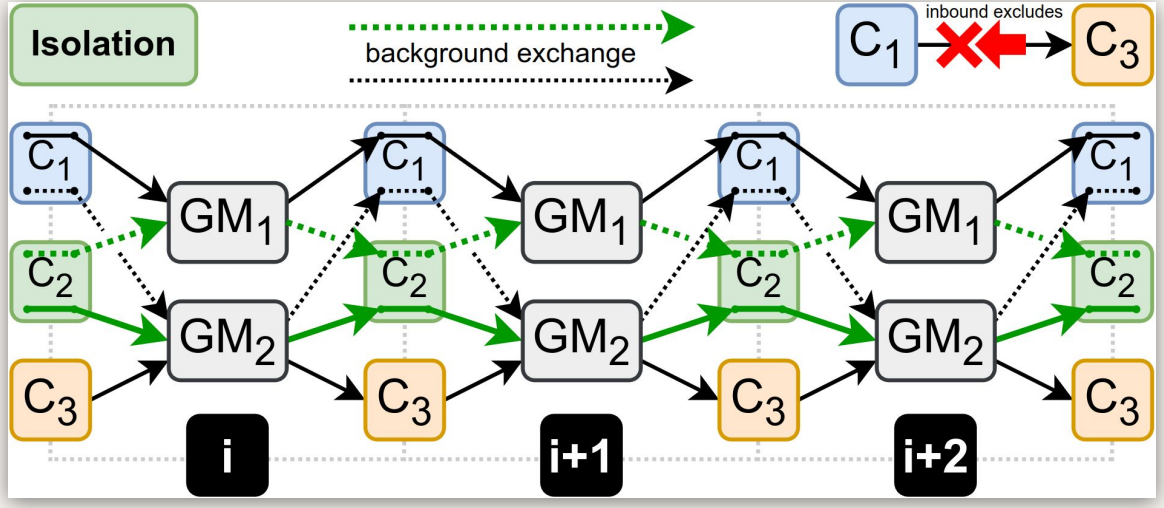
Robust resolution

- Solved through adding **background** model training
- C_1 and C_2 now train two models



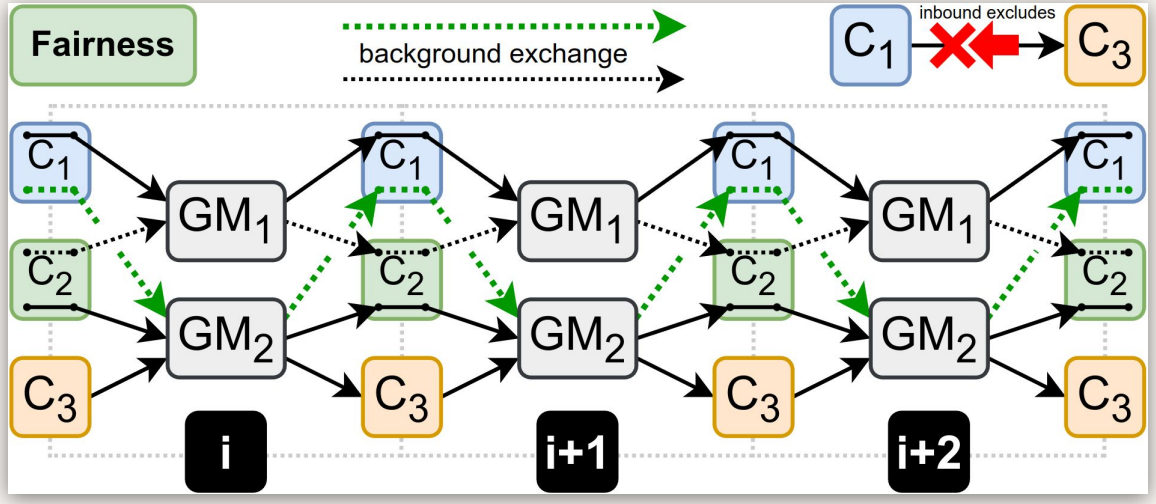
Robust resolution: isolation

- C_2 isolates its GM_1 and GM_2 training
  - GM_1 = background, for isolation
  - GM_2 = primary, for inference



Robust resolution: fairness

- Separating C_1's training to achieve fairness
  - GM_1 = primary, for inference
  - GM_2 = background, for fairness



RQ2: How can client-level disagreements be resolved?

- Through our robust multi-track resolution strategy
  - Prevents cross-contamination and ensures fairness

RQ3: How can temporal disagreements be accommodated?

- Clients specify a temporal scope (start/end)
- Therefore, we have to think about lifecycle strategies:
  - Initiating disagreements
    - Shallow: only exclude in future rounds
    - Deep: retroactively remove influence. E.g., unlearn or rewind
  - Lifting disagreements (**RQ3.1:** How can temporarily excluded clients rejoin?)
    - Shallow: client joins target track and adopts state
    - Deep: retroactively include client. E.g., merge or recompute

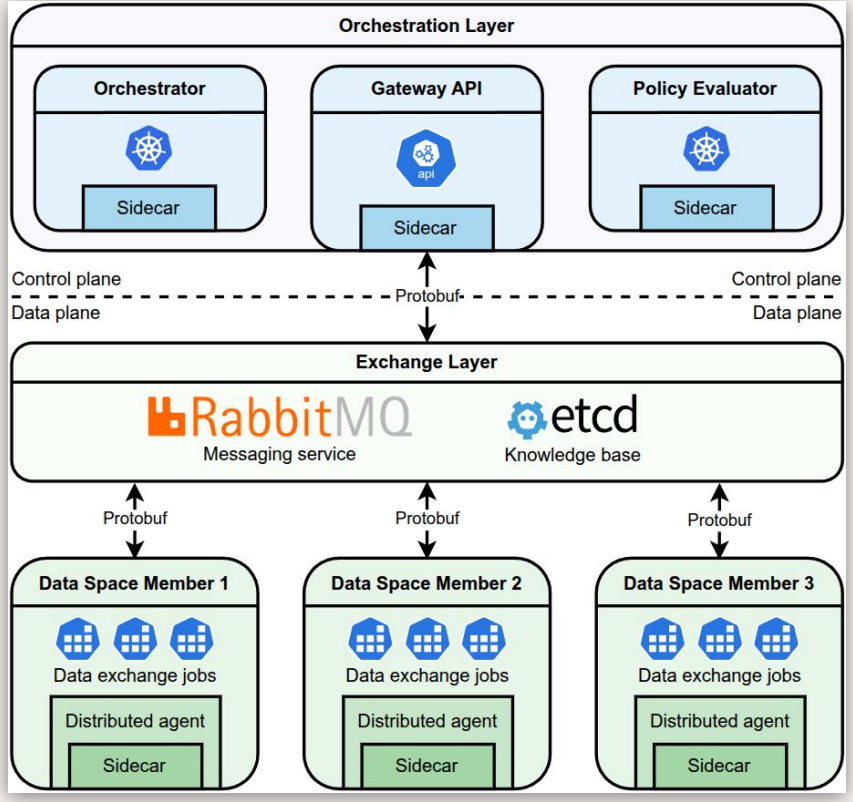
4

System design & implementation

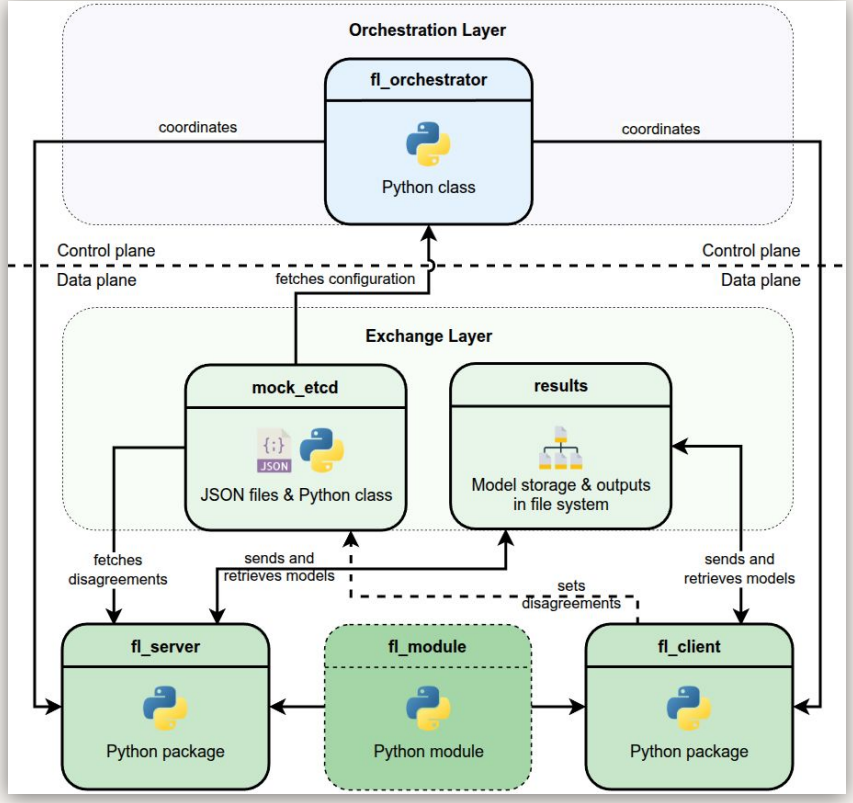
System design

- DYNAMOS: Dynamically Adaptive Microservice-based OS [2]
  - Policy-driven data exchange
  - Dynamic microservice composition
  - Initial goal was to implement our system in DYNAMOS
- We pivoted to a DYNAMOS-*inspired* system
  - DYNAMOS was not as flexible as hoped
  - Redeployment time was bottleneck for rapid prototyping

System design

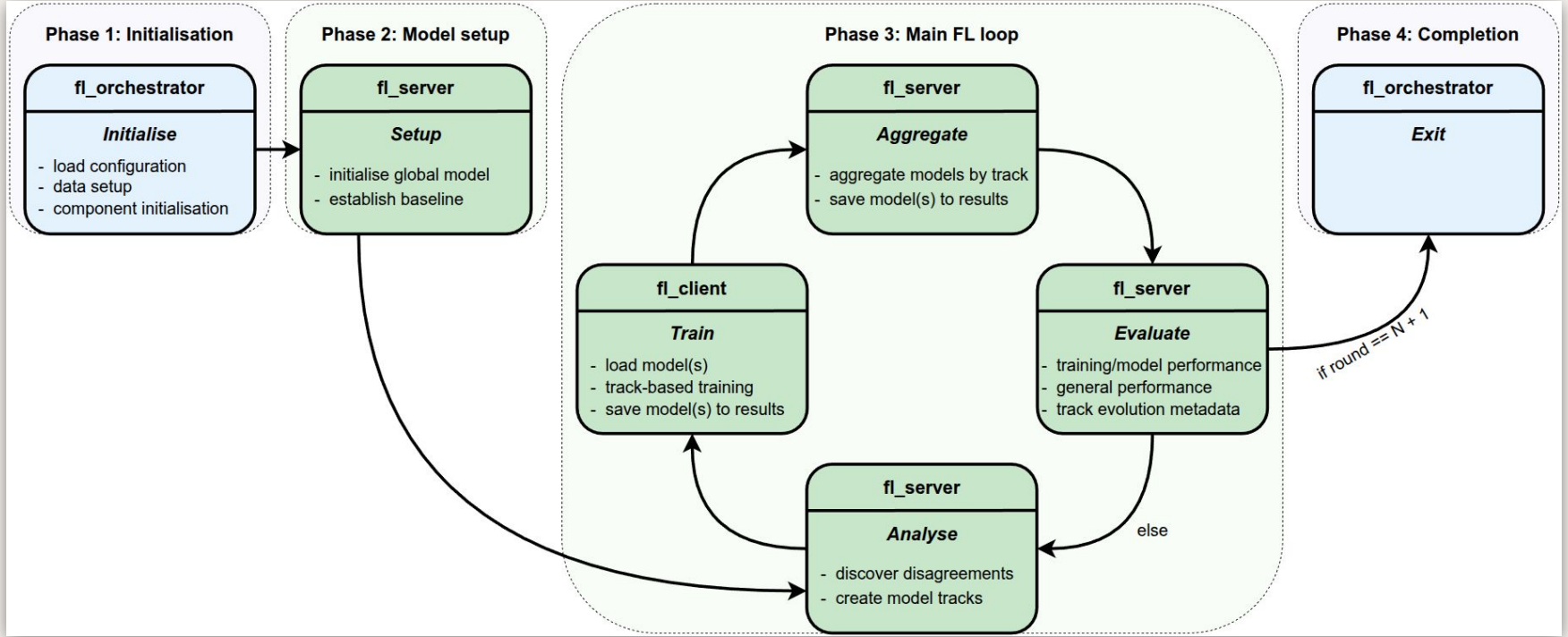


DYNAMOS [2] (improved visualisation)



Our DYNAMOS-inspired monolith

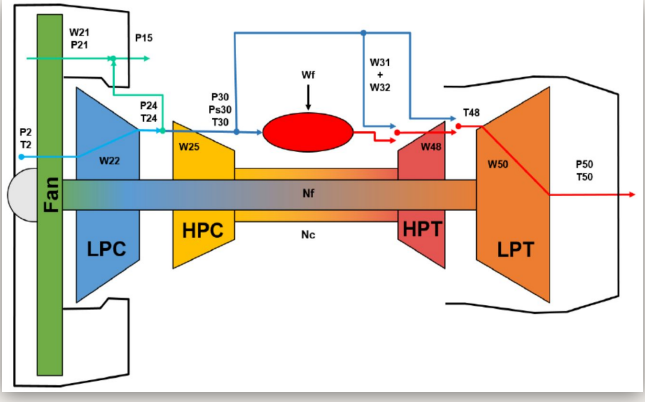
System flow


Phase 1: Initialisation
fl_orchestrator
Initialise
- load configuration
- data setup
- component initialisation
Phase 2: Model setup
fl_server
Setup
- initialise global model
- establish baseline
Phase 3: Main FL loop
fl_server
Aggregate
- aggregate models by track
- save model(s) to results
fl_client
Train
- load model(s)
- track-based training
- save model(s) to results
fl_server
Evaluate
- training/model performance
- general performance
- track evolution metadata
fl_server
Analyse
- discover disagreements
- create model tracks
else
if round == N + 1
Phase 4: Completion
fl_orchestrator
Exit

5
Experimental setup

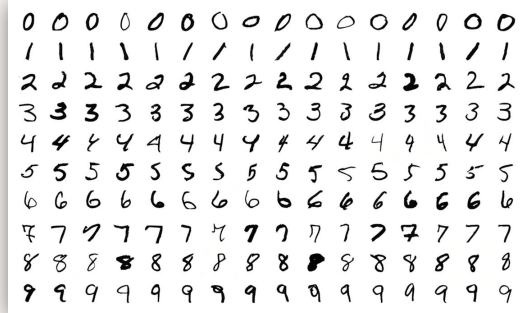
Dataset: N-CMAPSS [3]

- New Commercial Modular Aero-Propulsion System Simulation (N-CMAPSS)
- Developed and published by NASA
- Remaining Useful Life (RUL) prediction
- Motivation
  - Use in related theses [4]
  - Realistic non-IID time-series data
- Model architecture: Multilayer Perceptron



Dataset: MNIST [5]

- Modified National Institute of Standards and Technology (MNIST)
- Introduced by Yann LeCun
- Handwritten number prediction
- Motivation
  - Widely used within ML-world
  - Small size; ideal for prototyping
  - Ease of IID and non-IID partitioning
- Model architecture: Convolutional Neural Network



Scenarios

Scenarios	Description	Datasets
S1-S3	Permanent disagreements	Both
S4-S6	Temporary disagreements	Both
S7-S31	Scalability (5 sets)	Mostly MNIST
S32, S33	Submodel reuse	Both
S34	Deep mechanisms	Both

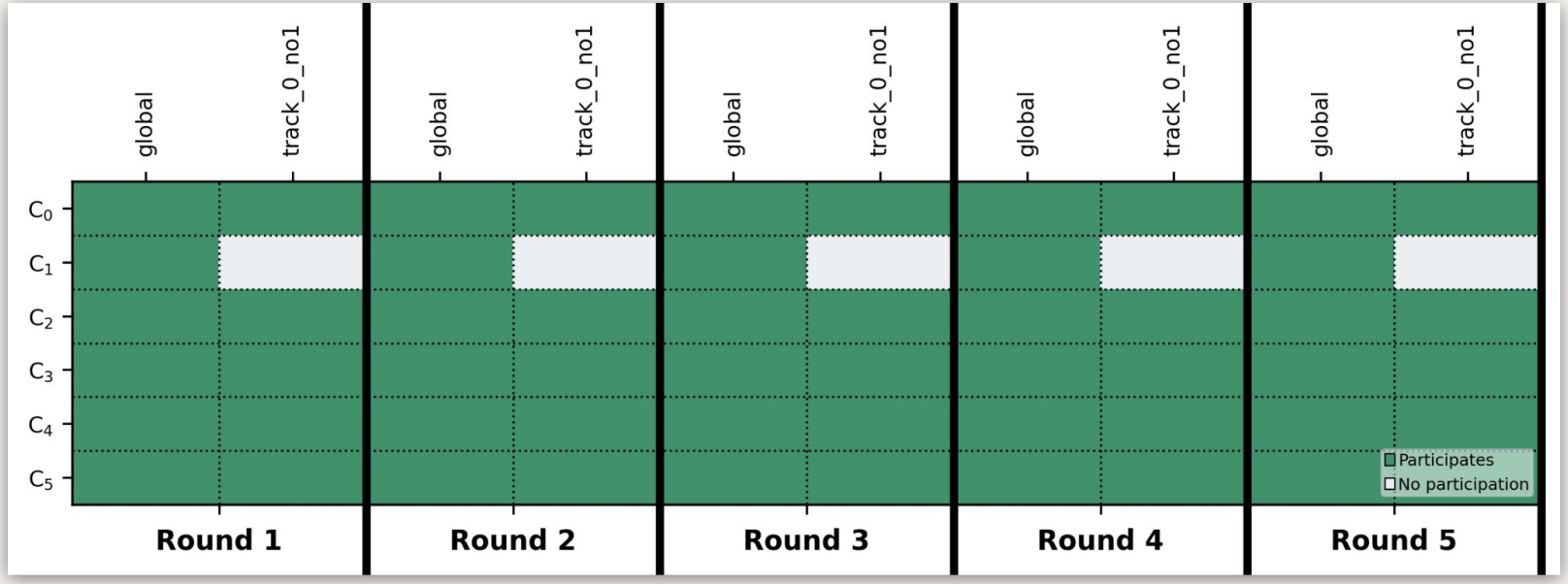
6

Results

Scenario 1 (permanent exclusion)

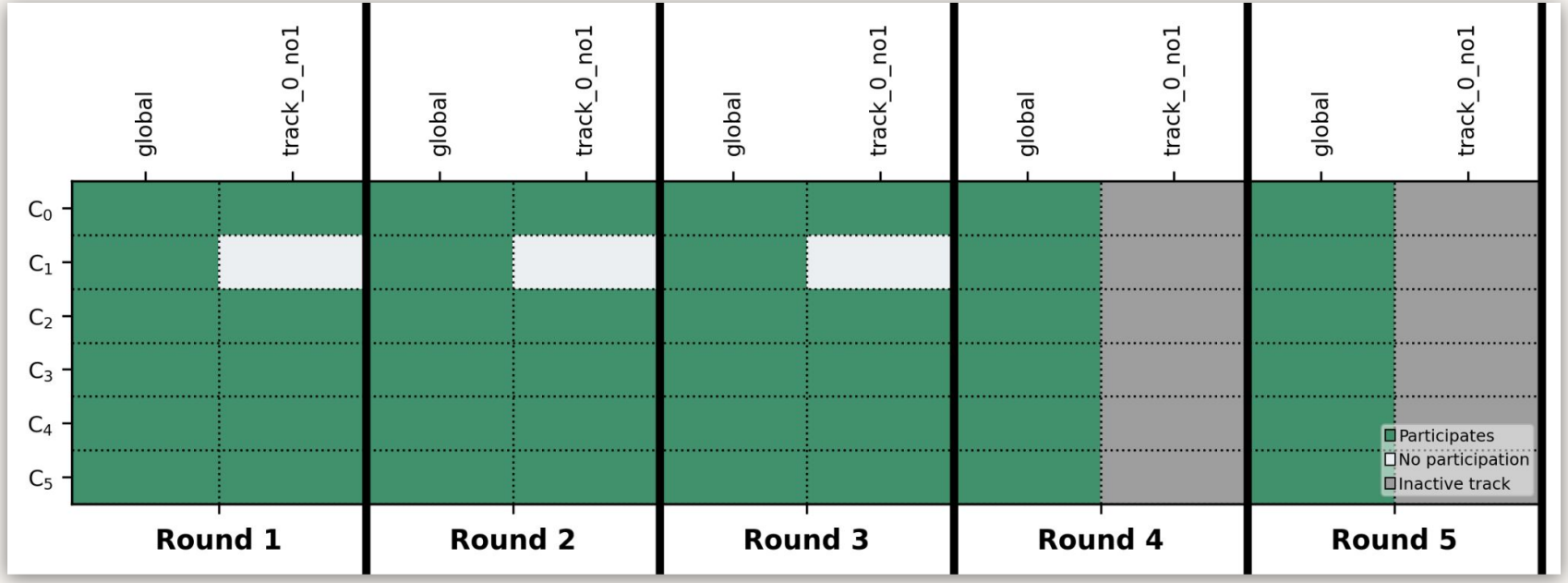
C_0 inbound excludes C_1, resulting in track_0_no1,

meaning primary track for C_0, excluding (no) C_1



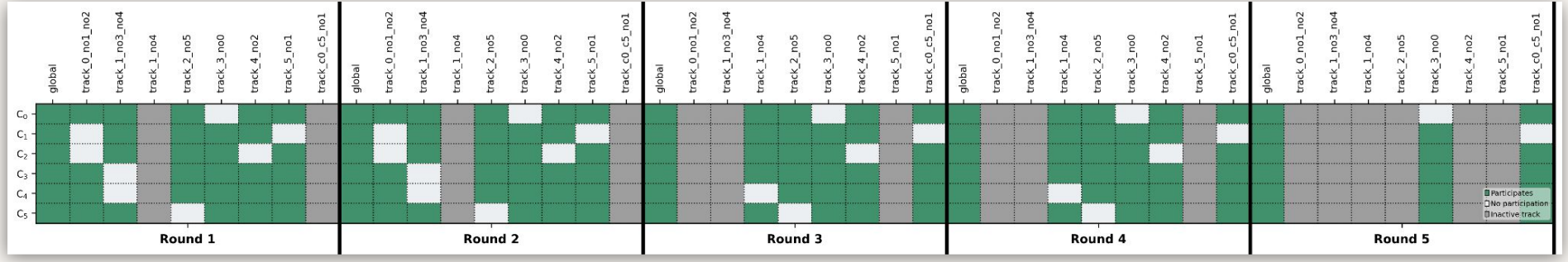
Scenario 4 (temporary exclusion)

C_0 inbound excludes C_1 until round 3, after which the system continues with regular FL



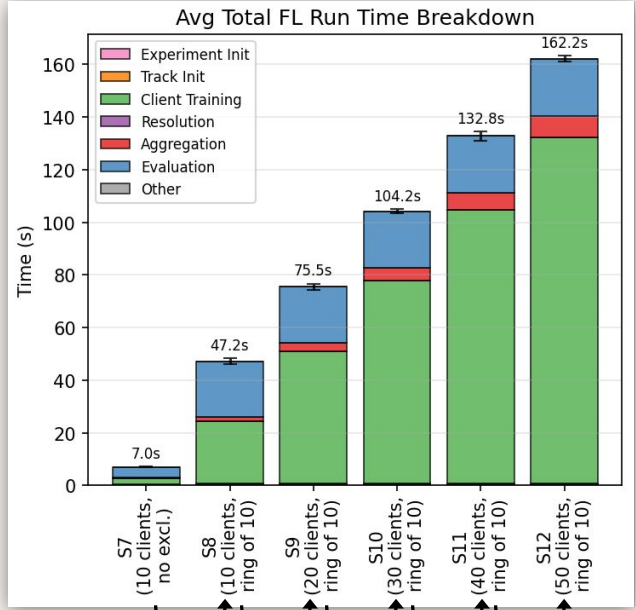
Scenario 6 (complex mix)

Track contribution visualisations are automatically generated for any scenario, even more complex ones



Light vs heaviest scalability test

Light scalability scenario set

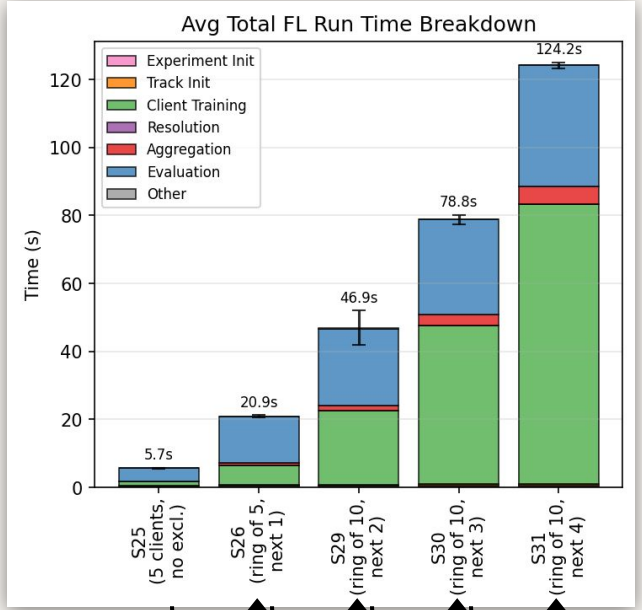


Each next scenario:

🚀 +10 clients

🔒 10 disagreeing clients

🔒 1 disagreement per disagreeing client



Heaviest scalability scenario set

Each next scenario:

🚀 +5 clients

🚀 +5 disagreeing clients

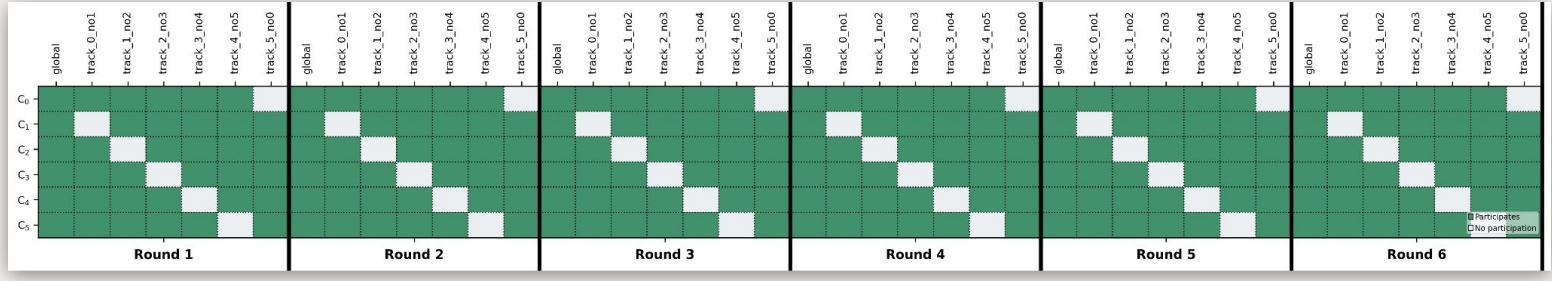
🚀 +1 disagreement per disagreeing client

Scalability results main takeaways

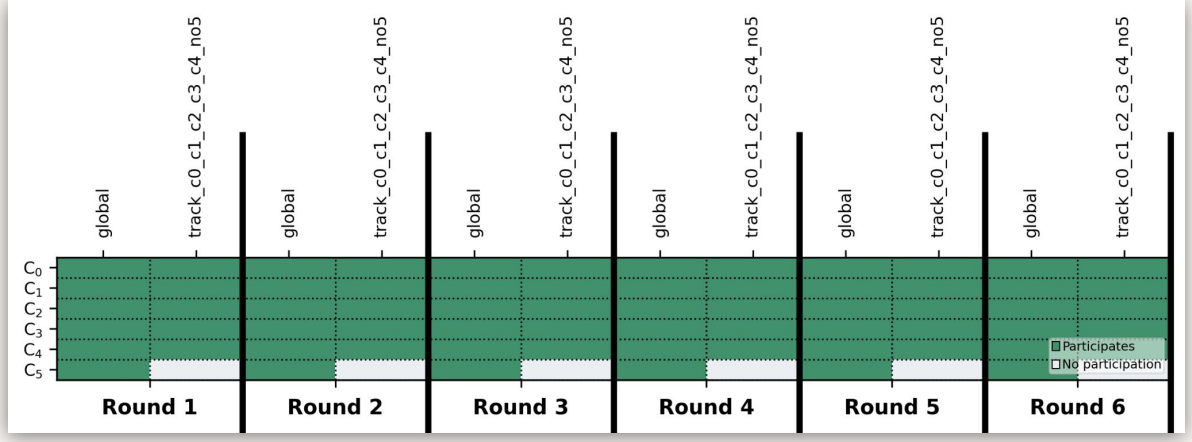
- For all scenario sets:
  - Resolution < 1ms per round
  - Aggregation < 2s per round
  - Client training time is largest
- Growth is
  - linear for lighter scenario sets
  - super-linear for heavier scenario sets
- N-CMAPSS performs the same as MNIST (for the one compatible set)
- Results are averaged across 5 runs
- Evaluation is arbitrarily large due to our extensive result generation

Submodel reuse (S32 & S33)

Each client excludes the next -> many different tracks



Each client excludes C_5 -> consolidated track



RQ2.1: How can sustainability be improved for similar disagreements?

- Implement a submodel reuse strategy; consolidating identical tracks
  - Reduces computational and storage overhead

RQ4: How can scalability challenges be accounted for?

- Use a highly efficient resolution algorithm
- Mitigate track growth with submodel reuse

7
Discussion

RQ2.2: How can granular access control help?

- Provides a formal language for data exclusions
- ABAC can exclude data based on attributes
  - (Attribute-Based Access Control, [6])
  - E.g., “Exclude all data originating from country X”
- FBAC can restrict specific operations on data
  - (Function-Based Access Control, [7])
  - E.g., “Forbid a `median` operation on this part of my data”

Limitations

- Centralised simulation ignores real-world network effects
- Descriptive track naming convention is not scalable
- Lifecycle operations limited to initiation and lifting

Threats to validity

- Limited scale and simulated environment threaten generalisability
- Implementation artifacts resulted in a self-inflicted CUDA memory bottleneck
- Conclusions rely on visual trends, not statistics (e.g., regression analysis)

Future work

- Develop verifiable unlearning and influence analysis methods
- Integrate confidential compute for trusted server execution
- Transition to a fully distributed, real-world implementation

8

Conclusion

Main contributions

- Exhaustive disagreement taxonomy
- Robust multi-track resolution strategy
- Efficient algorithms for resolution and aggregation
- Proof-of-concept with open-source reproducibility tools

Thank you for listening!

Try it out! https://github.com/DaanRosendal/fl-disagreement-resolution

References

[1] McMahan et al., "Communication-efficient learning of deep networks from decentralized data", arXiv preprint, 2016.

[2] Stutterheim et al., "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned", ICSA Companion, 2024.

[3] Chatterjee & Keprate, "Exploratory data analysis of the n-cmapss dataset for prognostics", IEEM 2021

[4] Dalgkitsis et al., "Secure Collaborative Model Training with Dynamic Federated Learning in Multi-Domain Environments", SC24 Workshops, 2024.

[5] LeCun et al., "Gradient-based learning applied to document recognition", Proceedings of the IEEE, 1998.

[6] Routh, A. K. & Ranjan, P., "A comprehensive review on granularity perspective of the access control models in cloud computing", IATMSI, 2024.

[7] Desmedt, Y. & Shaghaghi, A., "Function-based access control (FBAC): From access control matrix to access control tensor", SACMAT, 2020.